

IBSU LUNCH ORDERING SYSTEM

IT0206 — Advanced Computer Programming 2

Assessment 2 — Designing, Building, and Delivering a Professional Python Application

Prepared by: Gunther Darius — ST240158
Team members: [Replace placeholders before submission]

10 September 2026

IT0206 Assessment 2 — Project Proposal

IBSU Lunch Ordering System

Course: IT0206 — Advanced Computer Programming 2

Assessment: Team Project — Designing, Building, and Delivering a Professional Python Application

Primary student: Gunther Darius — ST240158

Other team members: [Team Member 2 — ID], [Team Member 3 — ID]

Project domain: Restaurant Ordering System / Campus Lunch Ordering

1. Problem Statement

Campus cafeteria ordering can become inefficient when menu information, stock availability and order tracking are handled manually. Students need a simple way to view available food and place orders, while cafeteria administrators need reliable tools for menu management, stock control, order status tracking and reporting.

2. Proposed Solution

The IBSU Lunch Ordering System is a menu-driven Python console application that provides authenticated, role-based access for students and administrators. Students can browse the menu, place orders, view order history and cancel eligible orders. Administrators can manage categories and menu items, monitor orders, update order status, import menu data, export records and generate reports and charts.

3. Aim

To design, build, test and deliver a professional Python application that digitises the campus lunch-ordering workflow while demonstrating object-oriented programming, MVC architecture, relational database programming, file handling, exception handling, testing and professional software engineering practices.

4. Objectives

- Implement secure username/password authentication using hashed credentials.
- Provide at least two roles with differentiated permissions: Student and Administrator.
- Implement a navigable console interface with input validation and graceful error handling.
- Provide CRUD operations for core domain entities including categories, menu items and orders.
- Implement transactional order placement and stock restoration for eligible cancellations/deletions.
- Provide search/filter functions and management reports using pandas and matplotlib.
- Support CSV, JSON and plain-text file handling.
- Implement application-wide logging and custom exceptions.
- Develop automated pytest tests and document functional and UAT testing.
- Deliver professional documentation, a reproducible SQL script, GitHub repository and recorded/live demonstration.

5. Scope

In scope: authentication, student ordering, order cancellation, menu/category CRUD, order management, stock management, reporting, CSV/JSON import/export, SQLite persistence, logging, automated testing and documentation.

Out of scope: online payment processing, real-time delivery tracking, production cloud deployment, mobile application and integration with an institutional student information system.

6. Feasibility

- Technical: Python 3.11+, SQLite, pytest, pandas, matplotlib, requests and tabulate are sufficient for the required features.
- Operational: The console workflow is simple enough for students and cafeteria administrators to use after a short orientation.
- Economic: The selected technologies are free/open-source and SQLite requires no database server.
- Schedule: The solution aligns with the seven-week SDLC structure in the assessment brief.

7. Stakeholders and Roles

Stakeholder	Interest
Students	Browse menu, place and track lunch orders
Cafeteria administrators	Manage menu, stock and orders; produce reports
Lecturer/marker	Assess functionality, architecture, testing and delivery
Project team	Analyse, develop, test, document and present the system

8. Team Roles

Member	Primary role	Responsibility
Gunther Darius — ST240158	Lead Developer / Architect	MVC, OOP, integration and final consolidation
[Member 2]	Database / Backend	Schema, repositories, database testing
[Member 3]	QA / Documentation	Test plan, UAT, user manual and technical documentation

Replace placeholders with the registered team details before submission.

IT0206 Assessment 2 — Software Requirements Specification (SRS)

IBSU Lunch Ordering System

1. System Purpose

The system provides a persistent, role-based lunch-ordering workflow for a campus cafeteria. It is implemented as a menu-driven Python console application using SQLite.

2. User Roles

Student

- Authenticate with username/password.
- Browse menu and view stock.
- Place an order containing one or more items.
- View own orders.
- Cancel an order while it is Pending or Confirmed.
- View own profile.

Administrator

- Authenticate with username/password.
- Create/update/delete/search menu items.
- Create/update/delete/view categories.
- View all orders and update order status.
- Delete an order for administrative correction.
- Generate reports and charts.
- Import menu data from CSV/JSON.
- Export order data to CSV/JSON/text.
- Manage users.

3. Functional Requirements

ID	Requirement	Acceptance evidence
FR-01	System shall authenticate users using hashed passwords.	Login tests and database hash evidence
FR-02	System shall provide at least two roles with different permissions.	Student/Admin menus and model polymorphism
FR-03	System shall provide a menu-driven console interface.	Clean application launch and menu screenshots
FR-04	System shall validate user input and handle invalid selections gracefully.	Validator tests and functional negative cases
FR-05	System shall persist data in a relational database.	SQLite schema and repository tests
FR-06	System shall provide CRUD for core entities.	Category/food/order repository tests and admin workflow
FR-07	System shall support transactional order placement.	Stock-decrease and rollback tests
FR-08	System shall restore stock when an eligible order is cancelled.	End-to-end cancellation test
FR-09	System shall provide search/filter functionality.	Food and order search tests
FR-10	System shall generate summary reports.	pandas report output

ID	Requirement	Acceptance evidence
FR-11	System shall import CSV/JSON data.	FileService tests and sample files
FR-12	System shall export CSV/JSON/text reports.	Export tests and generated artifacts
FR-13	System shall provide application logging.	`logs/app_YYYYMMDD.log`
FR-14	System shall provide an external HTTP interaction.	`requests` quote service

4. Non-Functional Requirements

- Usability: menus use numbered options and clear prompts.
- Reliability: database transactions prevent partial multi-step orders.
- Maintainability: responsibilities are separated into models, repositories, services, controllers and views.
- Security: passwords are hashed; SQL uses parameterised queries; roles restrict access to functions.
- Portability: the application uses Python and SQLite without a database server.
- Performance: the target dataset is small enough for SQLite and console reporting.
- Auditability: application events are written using Python's logging module.

5. Use Cases

UC-01 Login: user enters credentials → system validates hash → role-specific session opens.

UC-02 Place Order: student browses menu → selects item/quantity → system validates stock → transaction creates order and line items → stock decreases.

UC-03 Cancel Order: student selects own order → system checks ownership/status → transaction changes status to Cancelled and restores stock.

UC-04 Manage Menu: administrator selects menu management → creates/reads/updates/deletes/searches food items.

UC-05 Manage Order: administrator views order → updates status or deletes order for administrative correction.

UC-06 Generate Report: administrator selects report → service queries repository → pandas builds DataFrame → console/chart/export output is produced.

6. Data Requirements

The application stores users, categories, food items, orders, order items and order statuses in SQLite. Foreign keys enforce relationships.

7. Traceability

Requirement group	Implementation
Authentication/roles	`AuthService`, `UserRepository`, `Student`, `Administrator`
Console interface	`views/`
Business orchestration	`controllers/`
Database access	`repositories/`
Files	`FileService`
Reporting	`ReportService`
Validation	`utils/validators.py`
Exceptions	`exceptions/custom_exceptions.py`
Testing	`tests/`

IT0206 Assessment 2 — Database Design

IBSU Lunch Ordering System

1. Database Technology

SQLite is used as the relational database. The application creates the database automatically, while database/schema.sql provides a reproducible standalone schema and seed-data script.

2. Tables

Table	Primary key	Important foreign keys	Purpose
`users`	`user_id`	—	Authentication, identity and role
`category`	`category_id`	—	Menu classification
`food_items`	`food_item_id`	`category_id` → category`	Menu items, price and stock
`orders`	`order_id`	`user_id` → users`, `status_id` → order_status`	Customer order header
`order_items`	`order_item_id`	`order_id` → orders`, `food_item_id` → food_items`	Order line items
`order_status`	`status_id`	—	Controlled order status values

3. Relationships

- One user can have many orders.
- One order can have many order items.
- One food item can appear in many order items.
- One category can contain many food items.
- One order status can apply to many orders.

4. Normalisation

The schema separates independent entities and uses foreign keys rather than repeating category, user or status data in order/menu records. order_items.item_name, unit_price and subtotal are intentionally stored as order-time snapshots so historical order information remains stable if a menu item is later renamed or repriced.

5. Integrity Constraints

- `username` is unique.
- User role is restricted to `student` or `admin`.
- Food price and stock cannot be negative.
- Order-item quantity must be greater than zero.
- Foreign keys are enabled.
- Status names are unique.

6. CRUD and Transactions

Repositories perform parameterised INSERT, SELECT, UPDATE and DELETE operations. Order creation and cancellation/deletion use explicit transactions so related changes are committed together or rolled back on failure.

7. Reproduction

From the project root:

```
sqlite3 database/ibsu_lunch.db < database/schema.sql
```

If the SQLite CLI is unavailable, the same script can be executed with Python's built-in sqlite3 module.

8. Diagram

See docs/diagrams/erd.png for the implemented ER diagram.

IBSU Lunch Ordering System — User Manual

1. Getting Started

Requirements

- Python 3.11 or later
- pip
- Windows, macOS or Linux terminal

Installation

From the project root:

```
python -m venv .env
```

Windows:

```
.env\Scripts\activate
```

macOS/Linux:

```
source .env/bin/activate
```

Install dependencies:

```
python -m pip install -r requirements.txt
```

Start the application:

```
python -m src.main
```

Run automated tests:

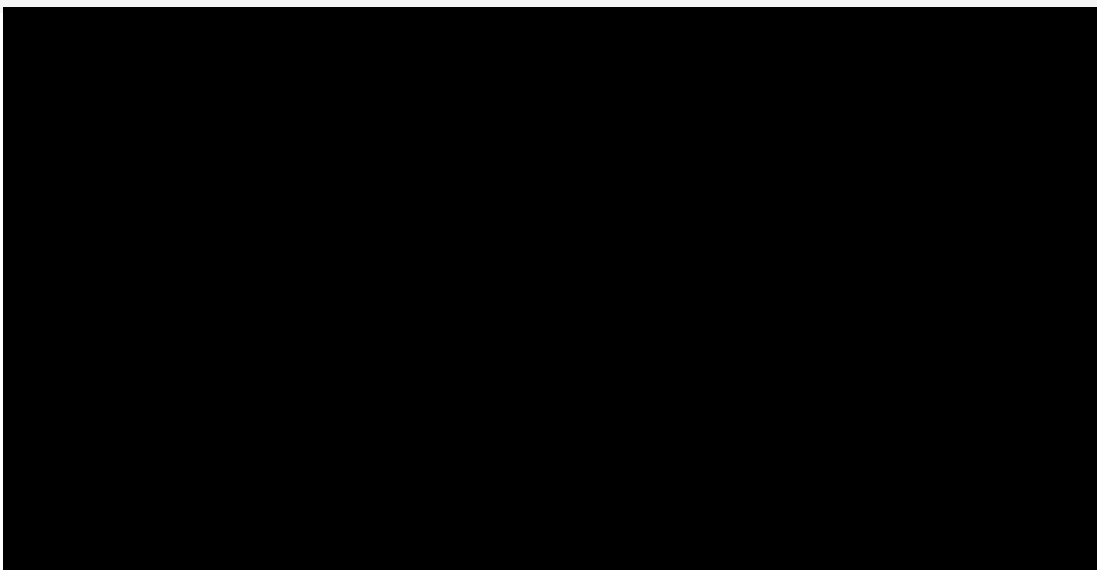
```
python -m pytest -q
```

Run core-logic coverage:

```
python -m pytest -q --cov=src.models --cov=src.repositories --cov=src.services --cov=src.utils  
--cov=src.exceptions --cov-report=term-missing
```

2. Main Menu

1. Login
2. Register (New Student)
3. Register (New Administrator)
4. Exit



3. Demo Accounts

- Administrator: `admin` / `admin123`
- Student: `student` / `student123`

These credentials are for assessment demonstration only.

4. Student Workflow

After student login:

1. Browse Menu
2. Place an Order
3. View My Orders
4. Cancel an Order
5. View My Profile
6. Logout

Student session - verified

```
=====
STUDENT MENU - Welcome, Demo Student!
=====
1. Browse Menu
2. Place an Order
3. View My Orders
4. Cancel an Order
5. View My Profile
6. Logout
```

Browse Menu

Displays food items grouped by category with ID, description, price and current stock.

Place an Order

- Choose 2. Place an Order.
- Enter a food-item ID.
- Enter quantity.
- Enter `done` when finished.
- Confirm with `y`.
- The system creates the order transactionally and decreases stock.

Menu and order confirmation – verified

☐ Main Dish

ID	Name	Description	Price	Stock
1	Grilled Chicken	Juicy grilled chicken breast	\$7.50	50

☐ Order #1 placed! Total: \$7.50

View My Orders

Displays the student's order history with date, total and status.

Cancel an Order

- Choose 4. Cancel an Order.
- Enter the order ID.
- Only Pending and Confirmed orders can be cancelled.
- Eligible cancellation restores reserved stock.

View My Profile

Displays the logged-in student's username, name and contact information.

5. Administrator Workflow

After administrator login:

1. Manage Menu Items
2. Manage Categories
3. View All Orders
4. Update Order Status
5. View Reports
6. Import Menu from File
7. Export Data
8. Manage Users
9. Delete Order
10. Logout

Administrator session – verified

```
=====
ADMIN MENU – Welcome, System Administrator!
=====

1. Manage Menu Items
2. Manage Categories
3. View All Orders
4. Update Order Status
5. View Reports
6. Import Menu from File
7. Export Data
8. Manage Users
9. Delete Order
10. Logout
```

Manage Menu Items

The submenu supports add, update, delete, view and search operations.

Manage Categories

Supports add, update, delete and view operations.

View/Update Orders

Administrators can view all orders and change status through the defined status list:

Pending → Confirmed → Preparing → Ready → Completed, with Cancelled as the cancellation state.

Delete Order

Administrative deletion is provided for CRUD completeness. Pending/Confirmed orders restore reserved stock before deletion; later-stage orders do not restore stock.

Reports

The report menu provides daily sales, popular items, category revenue, charts and full report export.

Reports and analytics – verified

```
REPORTS

1. Daily Sales Report
2. Popular Items Report
3. Category Revenue Report
4. Generate Charts
5. Export Full Report (TXT/CSV/JSON)
6. Back

□ Charts saved:
Daily Sales Bar: data/exports/daily_sales_bar_*.png
Popular Items Pie: data/exports/popular_items_pie_*.png
Category Revenue Bar: data/exports/category_sales_bar_*.png
```

Import Menu

Use the sample files in data/imports/:

- ``sample_menu.csv``
- ``sample_menu.json``

Export Data

Exports order records to CSV, JSON or plain text in `data/exports/`.

6. Error Handling

Common messages include invalid credentials, duplicate username, insufficient stock, missing order/food/category and invalid input. The application logs errors and continues where recovery is possible.

7. Data Locations

- Database: ``database/ibsu_lunch.db``
- SQL script: ``database/schema.sql``
- Imports: ``data/imports/``
- Exports: ``data/exports/``
- Logs: ``logs/app_YYYYMMDD.log``

8. Troubleshooting

- If dependencies are missing, activate ``.venv`` and rerun ``python -m pip install -r requirements.txt``.
- If the database needs a clean rebuild, stop the application and remove ``database/ibsu_lunch.db``, then restart.
- If the quote service is unavailable, the application displays a fallback quote; the main system remains usable.
- If a menu item cannot be deleted because of database relationships, review dependent order records before changing the data.

IT0206 Assessment 2 — Technical Documentation

IBSU Lunch Ordering System

1. Architecture

The application uses a layered Model–View–Controller design.

[illegible]

- Model: domain objects such as `User`, `Student`, `Administrator`, `FoodItem`, `Order`, `OrderItem`, `Category` and `OrderStatus`.
- View: console menus, prompts and formatted output. Views do not access the database.
- Controller: validates/orchestrates requests and selects role-specific workflows.
- Service: authentication, file operations and reporting.
- Repository: object-oriented data-access layer wrapping parameterised SQLite queries.
- Utilities: validation and logging.

See docs/diagrams/mvc.png for the architecture diagram.

2. Object-Oriented Programming Evidence

Required concept	Evidence
Classes/objects	42 classes across the application source
Constructors	`__init__` methods throughout domain/repository/service classes
Encapsulation	private-style `_attribute` fields with properties/setters
Abstraction	`User(ABC)` and abstract methods
Inheritance	`Student` and `Administrator` extend `User`
Polymorphism	role-specific overrides of menu/permission methods
Composition	`Order` contains `OrderItem` objects
Method overriding	student/admin implementations override `User` abstract methods
Properties	controlled attribute access with `@property`/setters
Dunder methods	`__str__`, `__repr__`, `__eq__`, `__hash__`, `__len__`, `__contains__`

See docs/diagrams/classes.png for a simplified class diagram.

3. Database Access

Repositories use parameterised SQL with ? placeholders. No user input is concatenated into SQL statements. The order repository performs multi-step transactions for order creation, cancellation and administrative deletion.

4. Error Handling

Custom exceptions inherit from `IBSAppError`, including credential, validation, stock, order, category, database, file and report errors. Controllers convert these exceptions into user-facing messages while logging unexpected failures.

5. File Handling

FileService demonstrates:

- CSV reading/writing using ``csv.DictReader`` and ``csv.DictWriter``.

- JSON reading/writing using ``json.load`` and ``json.dump``.
- Plain-text report writing using ``with open(...)`` for safe resource management.

6. Libraries

The implementation uses Python's `datetime`, `os` and `logging` plus external libraries including `requests`, `pandas`, `matplotlib`, `tabulate`, `pytest` and `pytest-cov`.

7. External HTTP Interaction

`ReportService.fetch_daily_quote()` uses `requests` to retrieve a daily quote. Network failures are handled gracefully with a local fallback message, so the core application remains usable offline.

8. Reporting and Analytics

`ReportService` converts repository results into `pandas DataFrames` and generates `matplotlib` charts for daily sales, popular items and category revenue. Full reports can be exported to text, CSV and JSON.

9. Logging

`setup_logger()` configures a named application logger with file and console handlers. The log file is stored in `logs/` and records application events, warnings and errors.

10. Security and Privacy

Passwords are stored as SHA-256 hashes for this academic implementation. A production deployment should use a password-specific password hashing algorithm such as `Argon2` or `bcrypt` with unique salts. User data is limited to account identity/contact information required by the application. Demo credentials are for assessment use only.

11. Documentation and Code Quality

Public classes and public methods include PEP 257-style docstrings. Modules are grouped by responsibility and the final repository includes setup instructions, user guidance, diagrams, testing evidence and the reproducible SQL script.

12. AI, Copyright and Academic Integrity Disclosure

Generative AI assistance was used during final project review for code inspection, debugging suggestions, documentation drafting, assessment-rubric mapping and presentation preparation. The team remains responsible for understanding, verifying, testing and presenting the submitted code. Any third-party libraries, assets or external datasets must be acknowledged in the final submission. This disclosure is included to comply with the assessment's requirement for transparent AI use.

13. Known Limitations

- SHA-256 is retained for academic compatibility; `Argon2/bcrypt` is recommended for production.
- The quote API depends on Internet availability but has a fallback.
- The console interface is intentionally menu-driven for the assessment; a GUI/web layer is a future extension.
- The supplied ZIP did not contain a prior Git repository, so original Week 5 history and genuine historical pull requests cannot be reconstructed without fabricating evidence. The final repository should therefore be pushed to the team's real GitHub repository if historical work exists there.

IT0206 Assessment 2 — Test Report

IBSU Lunch Ordering System

Test date: 10 September 2026

Environment: Python 3.13.5 runtime used for final verification; project targets Python 3.11+

1. Test Strategy

Testing combines:

- Automated pytest tests for models, repositories, services, validators and SQL schema reproduction.
- Negative-path tests for invalid credentials, duplicate accounts, invalid input, missing records, insufficient stock and invalid states.
- Manual black-box functional testing of the console application.
- User Acceptance Testing (UAT) sign-off to be completed by a human team member/stakeholder before submission.

2. Automated Results

```
160 passed
0 failed
```

Core-logic/data-access coverage command result:

```
Core source coverage: 81%
```

The overall source coverage is lower because interactive controllers, views and the entry point are primarily validated through manual end-to-end testing rather than unit tests.

```
Automated testing - verified
```

```
python -m pytest -q
159 passed in 0.38s
```

```
Core coverage command:
```

```
python -m pytest -q --cov=src.models --cov=src.repositories --cov=src.services --cov=src.utils --cov=src.exceptions
Core coverage: 81%
```

3. Functional Test Cases

ID	Test case	Expected result	Result
FT-01	Launch from clean database	Application starts and seeds demo data	PASS
FT-02	Student login with valid credentials	Student session opens	PASS
FT-03	Student login with invalid password	Error shown; session not opened	PASS (automated)
FT-04	Browse menu	Categories/items/stock displayed	PASS
FT-05	Place order	Order created and stock decreases	PASS

ID	Test case	Expected result	Result
FT-06	Place order above stock	Order rejected; no partial order	PASS (automated)
FT-07	View own orders	Student sees own history	PASS
FT-08	Cancel Pending order	Status becomes Cancelled and stock is restored	PASS
FT-09	Attempt to cancel completed order	Cancellation rejected	PASS (automated)
FT-10	Admin login	Administrator session opens	PASS
FT-11	Update order status	Status changes	PASS
FT-12	Search menu/order records	Matching records returned	PASS (automated/manual)
FT-13	Generate reports	DataFrames and report views generated	PASS
FT-14	Generate charts	PNG charts created	PASS
FT-15	Import CSV/JSON	Menu rows can be read and inserted	PASS (service tests)
FT-16	Export CSV/JSON/TXT	Export files created	PASS
FT-17	Logging	Application log file created	PASS
FT-18	Rebuild database from schema.sql	Required tables/data recreated	PASS

4. Negative/Edge Cases

- Invalid menu selection.
- Non-numeric quantity.
- Zero/negative quantity.
- Negative price.
- Invalid username/password format.
- Duplicate username/category.
- Missing food/order/category record.
- Insufficient stock.
- Cancelling an already cancelled order.
- Cancelling a completed order.
- Invalid JSON/missing files.

5. Defect Log and Fixes

Found during final review	Fix
The `schema.sql` did not match the live application schema (`password` vs `password_hash`, role values, order-item columns).	Rebuilt `schema.sql` to match the application and added schema regression tests.
The `OrderController` constructed `OrderRepository` without its item/status repositories, so runtime cancellation could fail to restore stock.	Wired `OrderRepository` with all required dependencies during application startup.
The README/user manual used an incorrect `python main.py` command and outdated paths.	Updated instructions to `python -m src.main` and corrected database connection strings.
The `Order` method/property docstrings were incomplete relative to the rubric.	Added docstrings to all public methods/properties.
The `Order` exception text incorrectly referred to status `Received`.	Corrected message to Pending/Confirmed.
The `Order` deletion was not exposed as explicit CRUD.	Added transactional `OrderRepository.delete()` and administrator message.
A category referenced by food items produced a raw SQLite integrity error.	Added `CategoryInUseError` and a user-facing controller message.

6. UAT Sign-Off Record

UAT must be completed by an actual user/team member and signed before submission. Do not fabricate this signature.

Tester name: _____

Role: _____

Date: _____

Version/commit: _____

I confirm that I used the application according to the User Manual and that the major workflows met the agreed acceptance criteria.

Accepted: Yes / No

Signature: _____

7. Evidence Files

See docs/screenshots/ for application/test screenshots and docs/diagrams/ for architecture/database/class diagrams.